PIPELINE DE DADOS COM IoT E DOCKER

Disruptive Architectures: IoT, Big Data e IA

Aluno(a): _____
Curso/Turma: _____
Data: ____/____/_____

Documento-modelo para completar após executar o projeto. Adicione capturas de tela reais e substitua os campos de análise pelos resultados observados no seu banco.

1. Contextualização

Dispositivos IoT podem gerar grandes volumes de leituras ambientais. Para que esses dados sejam úteis, é necessário recebê-los, validar sua qualidade, armazená-los e disponibilizar análises compreensíveis. Este projeto demonstra esse fluxo com um arquivo histórico de temperaturas, usando Python para ingestão, PostgreSQL para armazenamento e Streamlit/Plotly para visualização.

O arquivo do Kaggle é uma fonte histórica e não um fluxo de sensores em tempo real. Portanto, o trabalho simula a etapa de ingestão de dados IoT, mas não afirma que existe comunicação ao vivo com dispositivos.

2. Tecnologias utilizadas

Python e pandas: leitura, limpeza e padronização do CSV. SQLAlchemy e psycopg2: conexão entre Python e PostgreSQL. Docker Compose: execução reproduzível do banco em contêiner. SQL: criação de views agregadas. Streamlit e Plotly: dashboard interativo.

3. Etapas de implementação

1. Instalar Python, Git e Docker Desktop. 2. Criar e ativar um ambiente virtual e instalar requirements.txt. 3. Baixar o CSV do Kaggle e colocá-lo em data/. 4. Iniciar o PostgreSQL com docker compose up -d. 5. Executar python -m src.ingest para limpar e carregar os dados. 6. Executar sql/views.sql no PostgreSQL. 7. Iniciar streamlit run dashboard.py e validar os gráficos.

4. Views SQL

avg_temp_por_dispositivo: resume média, mínima, máxima e quantidade de leituras por dispositivo. Permite comparar sensores e investigar diferenças persistentes.

leituras_por_hora: conta quantas leituras ocorreram em cada hora do dia. Pode revelar períodos de maior atividade ou lacunas de coleta.

temp_max_min_por_dia: calcula máxima, mínima, média e quantidade de leituras por dia. Ajuda a observar variações diárias e amplitudes térmicas.

resumo_dataset: view auxiliar que informa total de leituras, dispositivos distintos e datas inicial/final do conjunto.

5. Dashboard e evidências

Cole aqui capturas reais obtidas após iniciar o dashboard. Inclua uma imagem geral e imagens legíveis dos gráficos. Não entregue esta seção com os espaços vazios.

[INSERIR CAPTURA 1: visão geral do dashboard e indicadores]

[INSERIR CAPTURA 2: média por dispositivo e leituras por hora]

[INSERIR CAPTURA 3: temperaturas máximas, mínimas e médias por dia]

6. Resultados e insights observados

Complete esta seção com valores que você realmente observar no dashboard:

• Total de leituras armazenadas: _____

• Quantidade de dispositivos: _____

• Dispositivo com maior média e valor: _____

• Hora com mais leituras e quantidade: _____

• Dia com maior amplitude térmica: _____

• Interpretação e possíveis causas: _____

7. Aplicação prática e melhorias

Em um ambiente real, a análise de temperaturas pode apoiar a identificação de condições ambientais fora do padrão, a comparação entre locais e a verificação da regularidade de sensores. Os resultados devem ser interpretados considerando a localização, calibração e frequência de cada dispositivo.

Como evolução, o CSV poderia ser substituído por ingestão contínua via MQTT ou HTTP; também poderiam ser adicionados alertas para limites de temperatura, autenticação, monitoramento do pipeline, testes automatizados e políticas de retenção.

8. Conclusão

O projeto integra ingestão e limpeza com Python, armazenamento relacional em PostgreSQL, execução do banco em Docker e análise visual com Streamlit. A conclusão final deve resumir os resultados observados após a execução, mencionar limitações e indicar como a solução poderia evoluir para dados em tempo real.